



A. MỤC TIÊU:

- Phân biệt được Stateless widget và Statefull widget
- Sử dụng được Row và Column để thiết kế giao diện
- Sử dụng được Text widget, Image widget, Button widget

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SV:

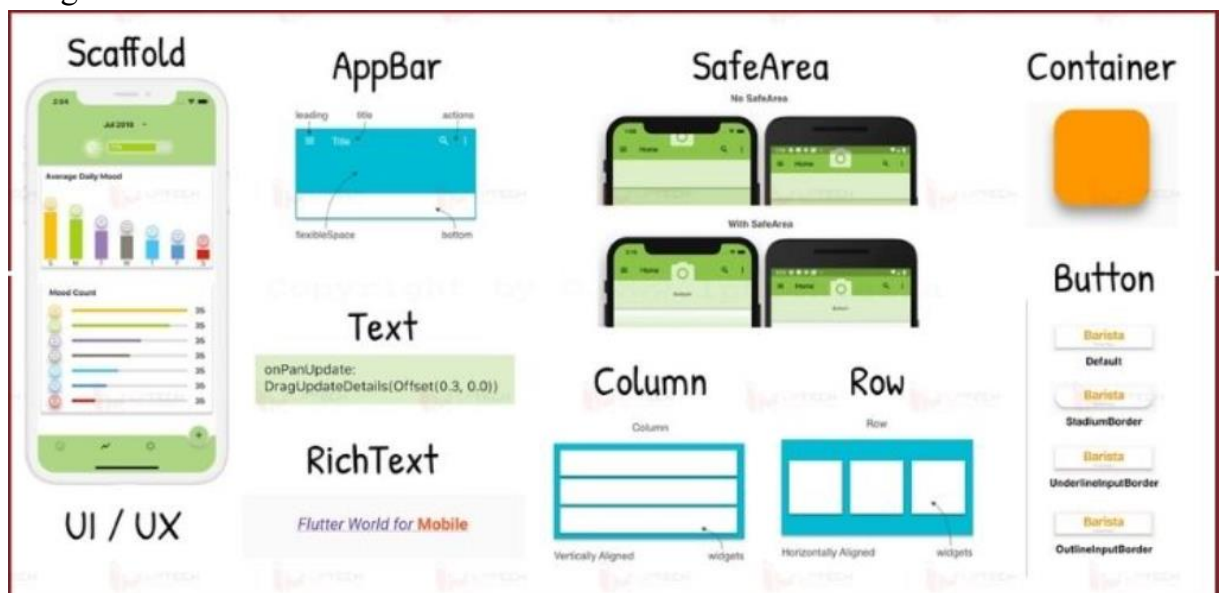
STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

a. Widget

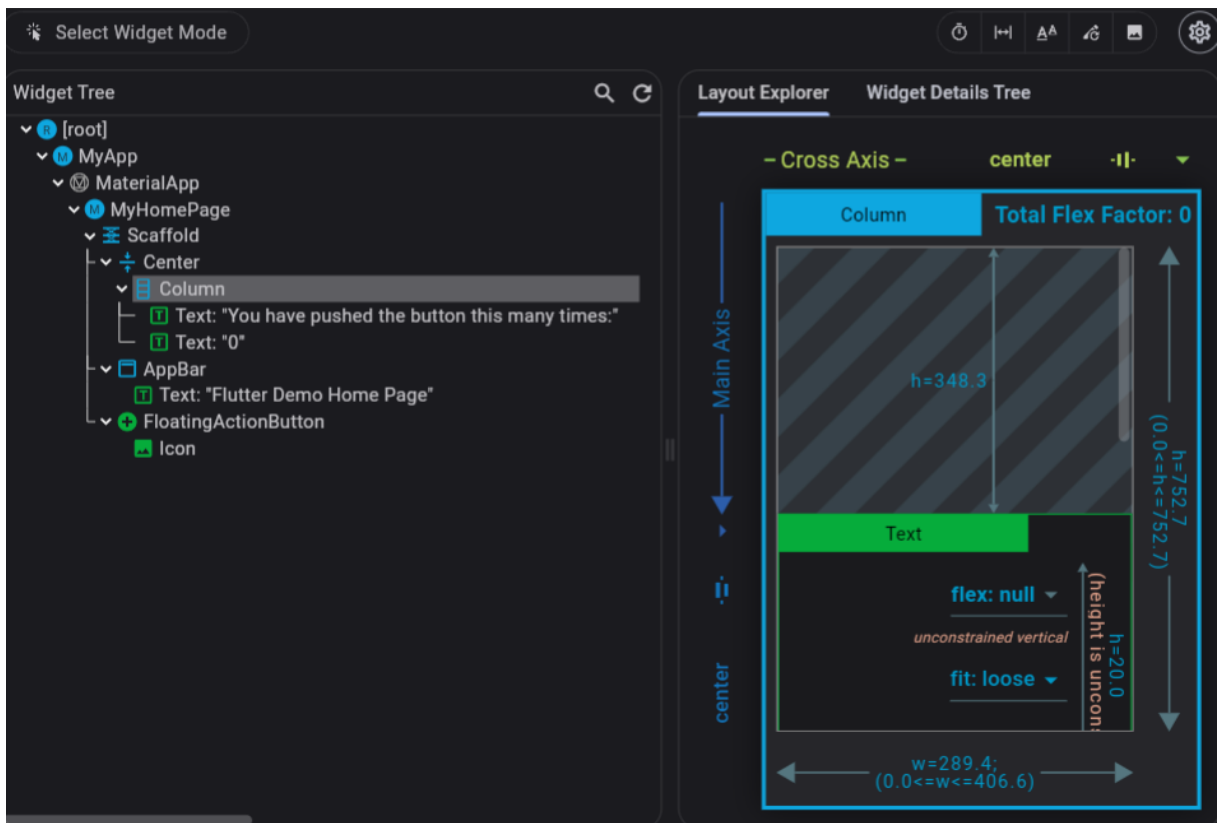
- **Khái niệm Widget** : Trong Flutter, **widget** là một khái niệm cốt lõi, đóng vai trò là các khối xây dựng cơ bản để tạo giao diện người dùng (UI). Mọi thứ trong Flutter đều là widget, từ các thành phần đơn giản như hiển thị văn bản, nút bấm, đến các thành phần phức tạp như danh sách, form, thậm chí toàn bộ màn hình. **Widget** trong Flutter là bất biến (immutable), nghĩa là chúng không thể thay đổi sau khi được tạo. Thay vào đó, khi cần thay đổi giao diện, Flutter sẽ tạo ra một widget mới thay thế widget cũ.



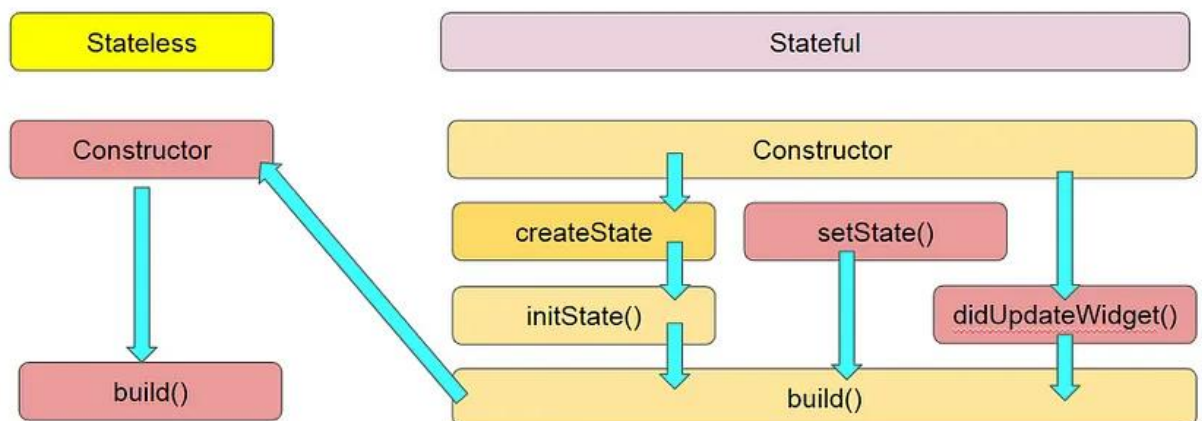
- **Có hai loại widget chính trong Flutter:**

- **Stateless Widget**: Là widget không lưu trữ trạng thái thay đổi. Ví dụ: một biểu tượng, hình ảnh hoặc một dòng chữ, một đoạn văn bản.

- **Stateful Widget:** Là widget có khả năng lưu trữ trạng thái có thể thay đổi.
Ví dụ: Một button có thể thay đổi màu sắc khi được nhấn, hoặc một bộ đếm tăng giá trị khi người dùng tương tác, Checkbox hoặc Slider,...
- **Các loại widget thông dụng**
 - **Widget bố cục (Layout Widgets):** Dùng để sắp xếp các widget con theo một cách cụ thể.
 - Row: Sắp xếp các widget con theo chiều ngang.
 - Column: Sắp xếp các widget con theo chiều dọc.
 - Stack: Chồng các widget con lên nhau.
 - Container: Một hộp chứa có thể tùy chỉnh kích thước, màu sắc, padding, v.v.
 - **Widget hiển thị (Display Widgets):** Dùng để hiển thị nội dung.
 - Text: Hiển thị văn bản.
 - Image: Hiển thị hình ảnh.
 - Icon: Hiển thị biểu tượng.
 - **Widget tương tác (Interactive Widgets):** Dùng để tương tác với người dùng.
 - ElevatedButton: Nút bấm.
 - TextField: Ô nhập liệu.
 - Checkbox: Hộp kiểm.
 - Slider: Thanh trượt.
 - **Widget chức năng (Functional Widgets):** Cung cấp các chức năng đặc biệt.
 - GestureDetector: Nhận diện các cử chỉ như chạm, vuốt.
 - FutureBuilder: Xây dựng UI dựa trên kết quả của một Future.
 - StreamBuilder: Xây dựng UI dựa trên dữ liệu từ một Stream.
- **Đặc trưng widget**
 - **Tính linh hoạt:** Widget cho phép xây dựng giao diện người dùng một cách linh hoạt và dễ dàng.
 - **Tái sử dụng:** các widget tùy chỉnh và tái sử dụng chúng trong nhiều phần của ứng dụng.
 - **Tính hiệu quả:** Cơ chế render hiệu quả, chỉ xây dựng lại các widget cần thiết khi trạng thái thay đổi.
- **Quản lý widget trong Flutter:** Các widget được tổ chức thành cây, nơi một widget có thể chứa nhiều widget khác gọi là widget tree.



- **Vòng đời của widget:** Mỗi widget trong Flutter có một vòng đời rõ ràng mà hiểu biết về nó có thể giúp bạn quản lý tài nguyên và hiệu năng tốt hơn. Đối với Stateless Widgets, vòng đời đơn giản chỉ bao gồm việc tạo và hủy.



- *createState()*: dùng để khởi tạo Stateful State và chỉ gọi một lần.
- *mounted = true*: sau khi State được khởi tạo thành công, BuildContext sẽ được assigned vào State. Điều đó nghĩa là Widget đã được insert vào Widget Tree. Ngược lại khi *mounted = false*, Widget đã remove ra khỏi Widget Tree.
- *initState()*: được gọi 1 lần sau khi state được khởi tạo.
- *build()*: được gọi nhiều lần, return về một widget (mình hay gọi nó là funtion để render UI).
- *setState()*: developer gọi hàm này để render lại UI khi state có sự thay đổi. Việc này đồng thời sẽ trigger *build()*.
- *didUpdateWidget()*: được gọi khi các arguments của constructor thay đổi.

b. Text widget

- Đặc điểm:

- Sử dụng để hiển thị một chuỗi văn bản
- Hỗ trợ nhiều thuộc tính khác nhau để định dạng văn bản như font, kích thước, màu sắc, độ đậm, kiểu chữ, v.v.
- Có thể được căn chỉnh theo các hướng khác nhau và có thể đặt độ giãn dòng.

- Một số thuộc tính

- data (bắt buộc): Là nội dung chuỗi văn bản cần hiển thị
- style: định dạng kiểu chữ thông qua lớp TextStyle với các thuộc tính sau:
 - color: Màu sắc văn bản.
 - fontSize: Kích thước chữ.
 - fontWeight: Độ đậm (ví dụ: `FontWeight.bold`).
 - fontStyle: Kiểu chữ (ví dụ: `FontStyle.italic`).
 - fontFamily: Tên font chữ.
 - decoration: Gạch chân, gạch ngang.
- textAlign: Căn chỉnh văn bản trong không gian hiển thị, có thể nhận các giá trị: `TextAlign.left`, `TextAlign.center`, `TextAlign.right`, `TextAlign.justify`.
- maxLines: Giới hạn số dòng hiển thị văn bản
- overflow: Xử lý khi văn bản vượt quá số dòng qui định trong thuộc tính maxLines, có thể nhận các giá trị:
 - `TextOverflow.ellipsis`: thêm dấu vào cuối khi vọt quá maxLines
 - `TextOverflow.clip`: cắt ngắn văn bản
 - `TextOverflow.visible`: hiển thị toàn bộ văn bản
 - `TextOverflow.fade`: làm mờ văn bản khi vượt quá maxLines
- textScaleFactor: - Tỷ lệ phóng to/thu nhỏ văn bản.

- Lưu ý

- Text widget là bất biến: Không thể thay đổi trực tiếp nội dung hoặc kiểu dáng sau khi tạo. Để cập nhật, cần phải tạo một Text widget mới.
- Hiệu suất: Text widget được tối ưu hóa để hiển thị nhanh, ngay cả với văn bản dài.
- Font chữ tùy chỉnh: Thêm font vào thư mục ``assets/fonts`` và khai báo trong `pubspec.yaml`.

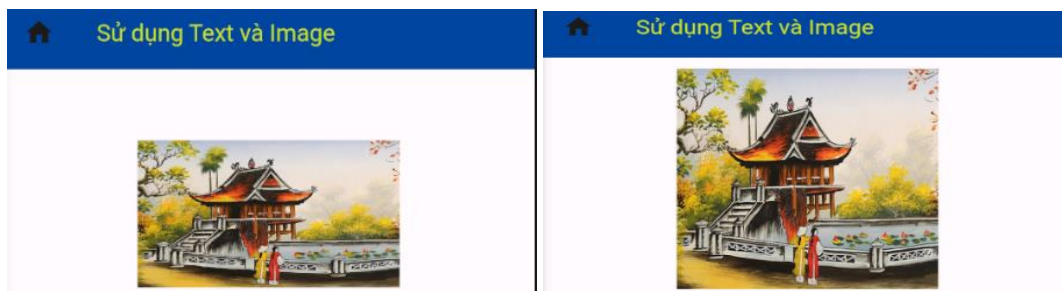
- Ví dụ: Sử dụng Text widget để hiển thị thông tin như sau

```
Text(  
  "LẬP TRÌNH DI ĐỘNG KHÓA 13!",  
  style: TextStyle(  
    color: Color.fromARGB(255, 235, 19, 19),  
    fontSize: 25,  
    fontWeight: FontWeight.bold),  
  textAlign: TextAlign.center)
```



c. Image widget

- **Đặc điểm:** Được sử dụng để hiển thị hình ảnh từ nhiều nguồn khác nhau như asset, mạng (URL), file, hoặc dữ liệu nhị phân.
- **Phân loại :**
 - Image.asset: Hiển thị hình ảnh từ thư mục **assets** của ứng dụng.
 - Thêm hình ảnh vào thư mục **assets/images** trong dự án.
 - Khai báo tài nguyên trong **pubspec.yaml**:
 - Sử dụng trong code:
 - Image.network: Tải hình ảnh từ URL.
 - Image.file: Hiển thị hình ảnh từ file trên thiết bị.
 - Image.memory: Hiển thị hình ảnh từ dữ liệu nhị phân (byte array).
- **Một số thuộc tính**
 - width và height: Điều chỉnh kích thước hình ảnh:
 - fit: Xác định cách co giãn hình ảnh, có thể nhận các giá trị:
 - *BoxFit.cover*: Phủ kín khung, có thể cắt xén.
 - *BoxFit.contain*: Hiển thị toàn bộ hình ảnh, không cắt xén.
 - *BoxFit.fill*: Kéo giãn hình ảnh để lấp đầy khung.
 - color và colorBlendMode: Pha trộn màu sắc với hình ảnh.
 - alignment: Căn chỉnh vị trí hình ảnh trong khung.
 - errorBuilder: Xử lý lỗi khi tải hình ảnh:
- **Lưu ý:**
 - Tối ưu hiệu suất: Sử dụng `cached_network_image` để cache hình ảnh từ mạng.
 - Kích thước hình ảnh: Luôn xác định width và height để tránh lỗi layout.
 - Xử lý lỗi: Luôn thêm `errorBuilder` để ứng dụng không crash khi tải hình ảnh thất bại.
- **Ví dụ:** Sử dụng Text widget để hiển thị thông tin như sau



- Khai báo các file sử dụng trong tập tin `pubspec.yaml` các tập tin cần sử dụng

```
assets:
  - assets/images/logo_huit_01.png
```

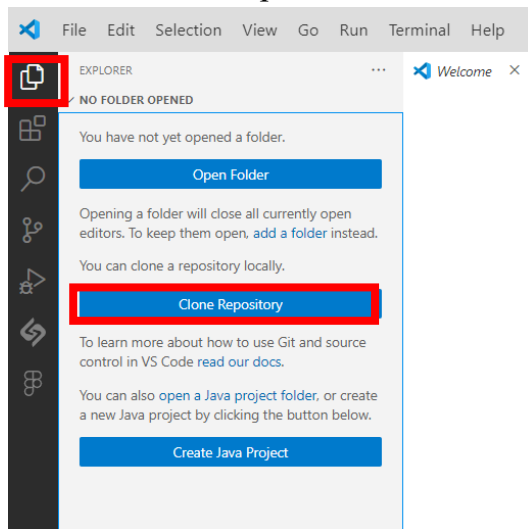
Sử dụng và hiển thị hình ảnh sử dụng Image widget

```
Image.asset("assets/images/chua_mot_cot.jpg",  
            width: 200, height: 200, fit: BoxFit.scaleDown)  
  
//hoặc  
Image.asset("assets/images/chua_mot_cot.jpg",  
            width: 200, height: 200, fit: BoxFit.fill)
```

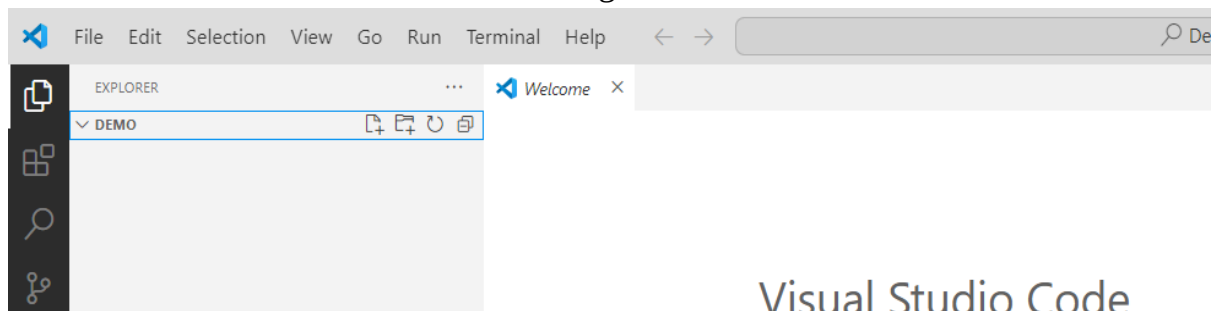
2. Bài tập hướng dẫn

a. Các bước để tạo một ứng dụng flutter

- Mở Visual code, chọn biểu tượng (Explorer)  hoặc bấm tổ hợp phím Ctrl + Shift + E. Chọn Open Folder để chọn thư mục làm việc

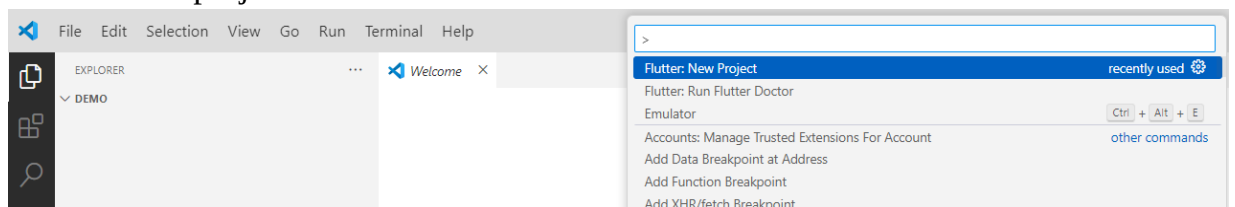


- Sau khi chọn thư mục làm việc ta được giao diện như sau:

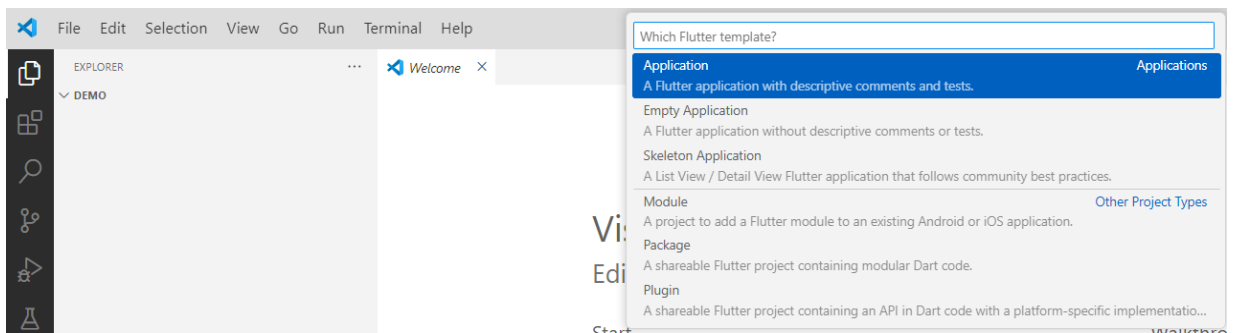


Visual Studio Code

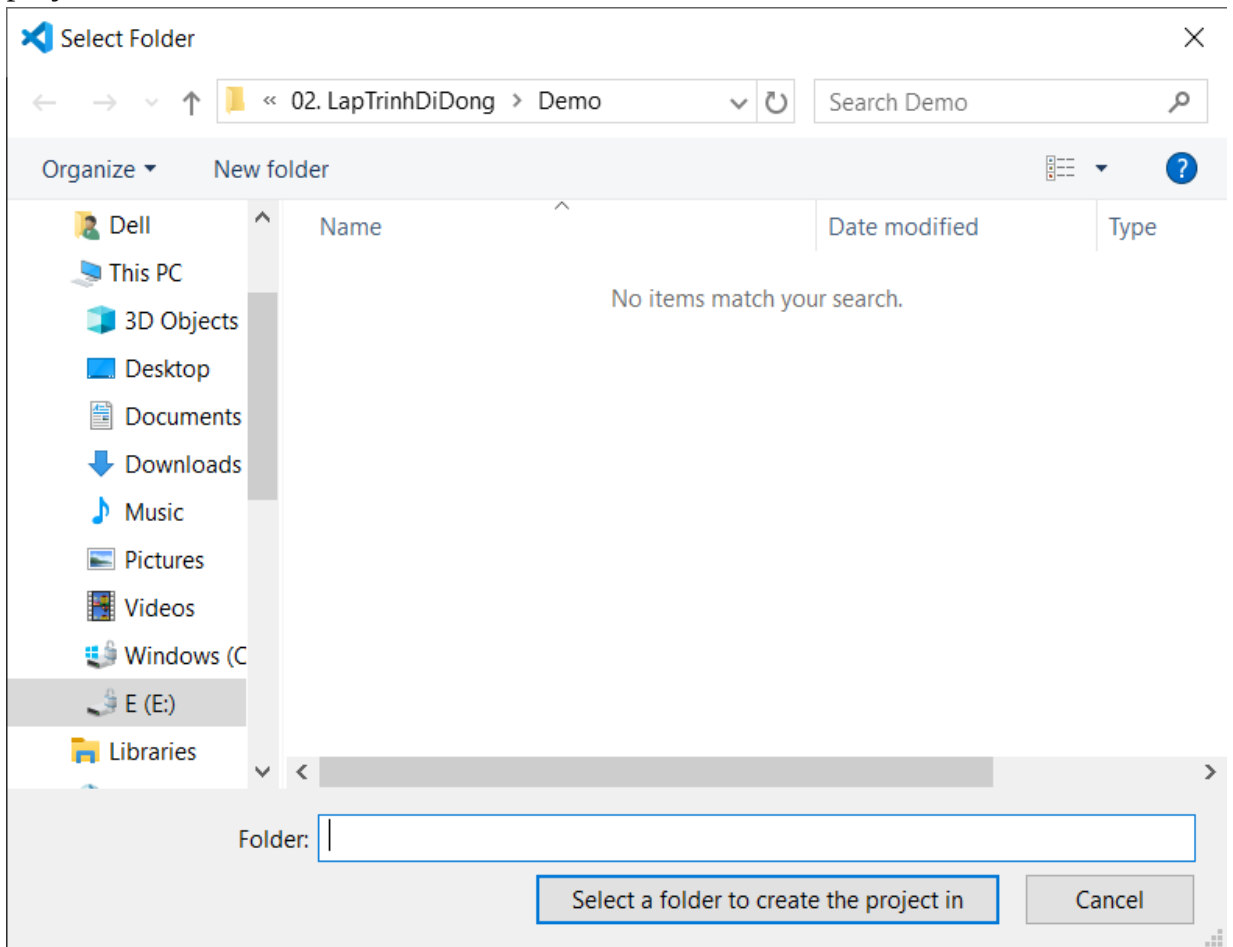
- Để tạo ứng dụng flutter ta chọn View -> Command Palette (Ctrl + Shift + P) -> Flutter: New project



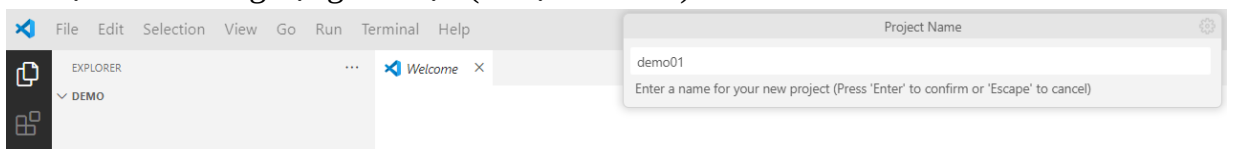
- Chọn Application để tạo ra một project chuẩn theo cấu trúc lưu trữ của flutter



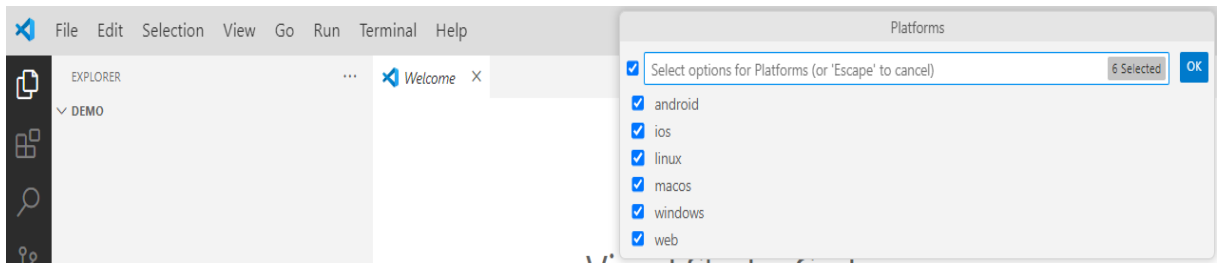
- Chọn thư mục muốn lưu trữ ứng dụng của mình -> Select a folder to create the project in.



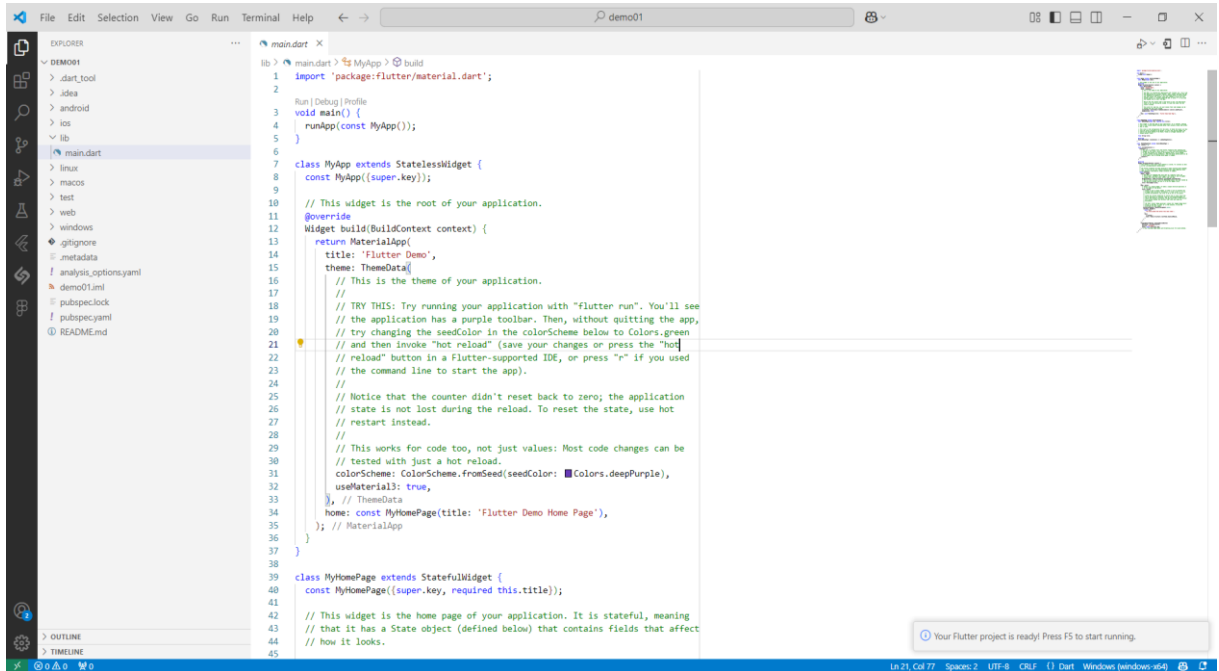
- Đặt tên cho ứng dụng của bạn (ví dụ: demo01).



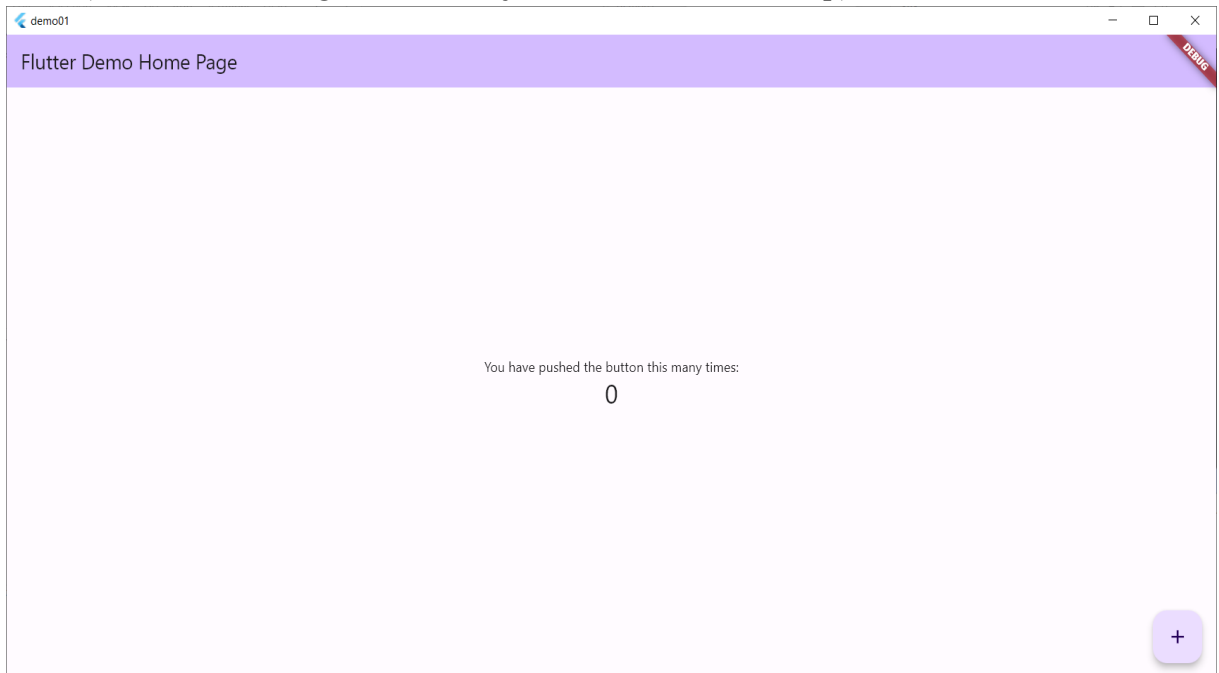
- Để có thể chọn platforms cho project, bấm vào biểu tượng , sau đó chọn Platforms all. Tiếp theo, bạn hãy chọn các Platforms mà ứng dụng cần có, sau đó bấm OK. Tiếp tục bấm OK để hoàn thành quá trình tạo ứng dụng bằng flutter.



- Kết quả thu được như sau:



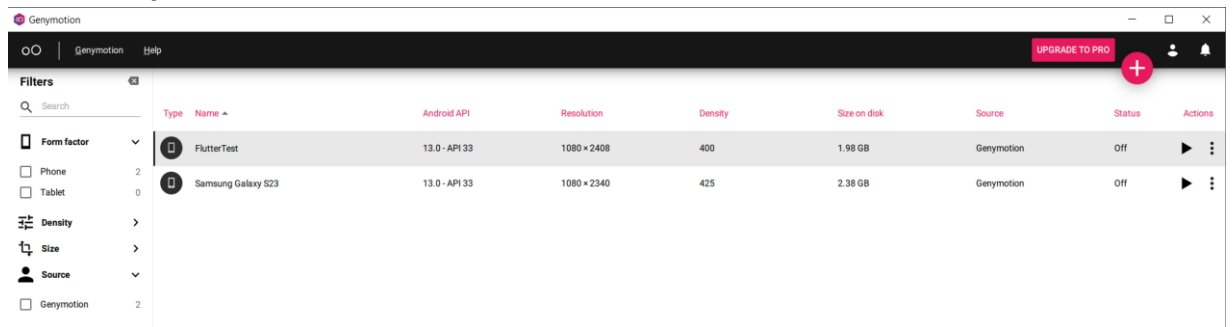
- Để thực thi chương trình, bấm Run -> Run without Debugging để chạy chương trình (mặc định chương trình sẽ chạy trên Windows desktop)



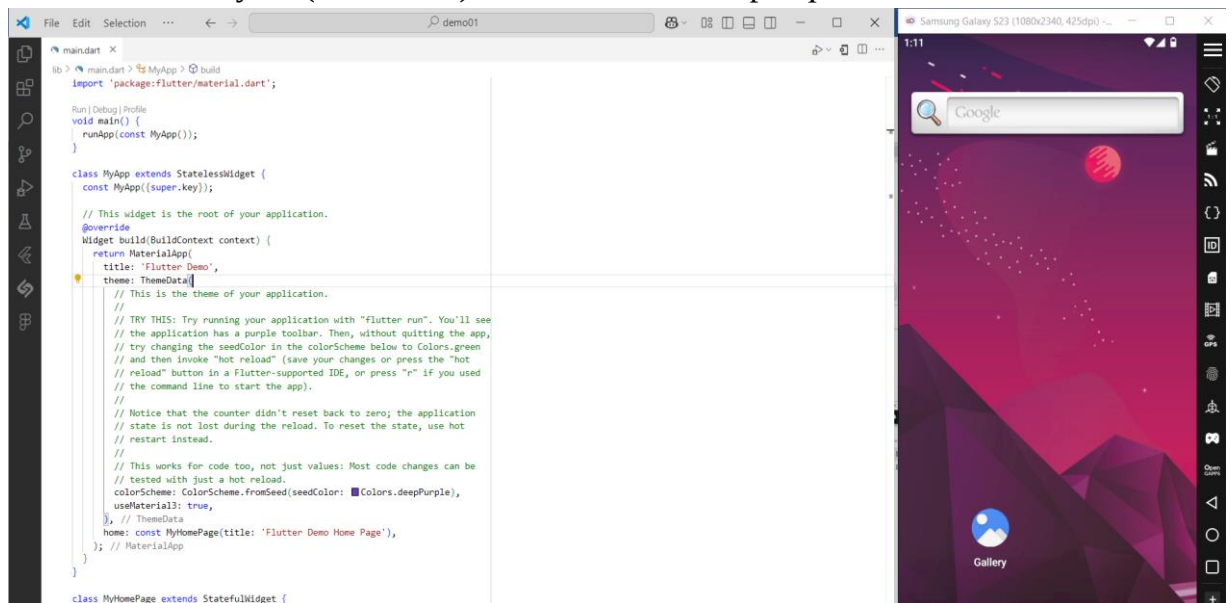
b. Tạo máy ảo trên Genymotion và thực thi.

- Tải và cài đặt Genymotion (Phiên bản Personal). Lưu ý bạn cần đăng ký tài khoản để có thể sử dụng Genymotion

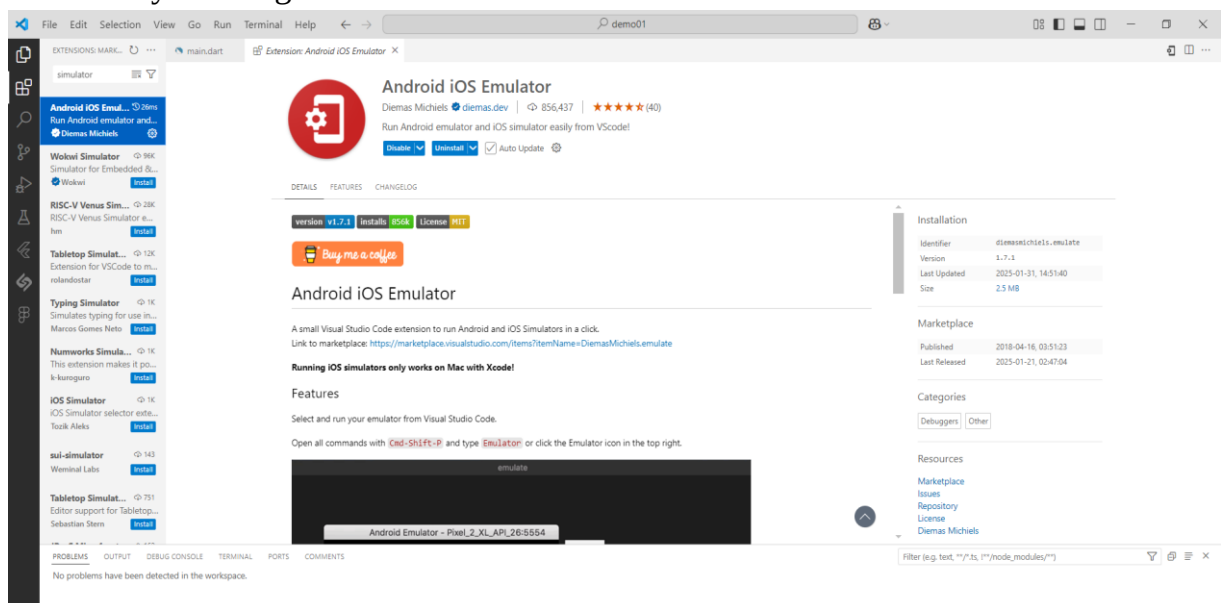
- Tạo máy ảo. Ví dụ:



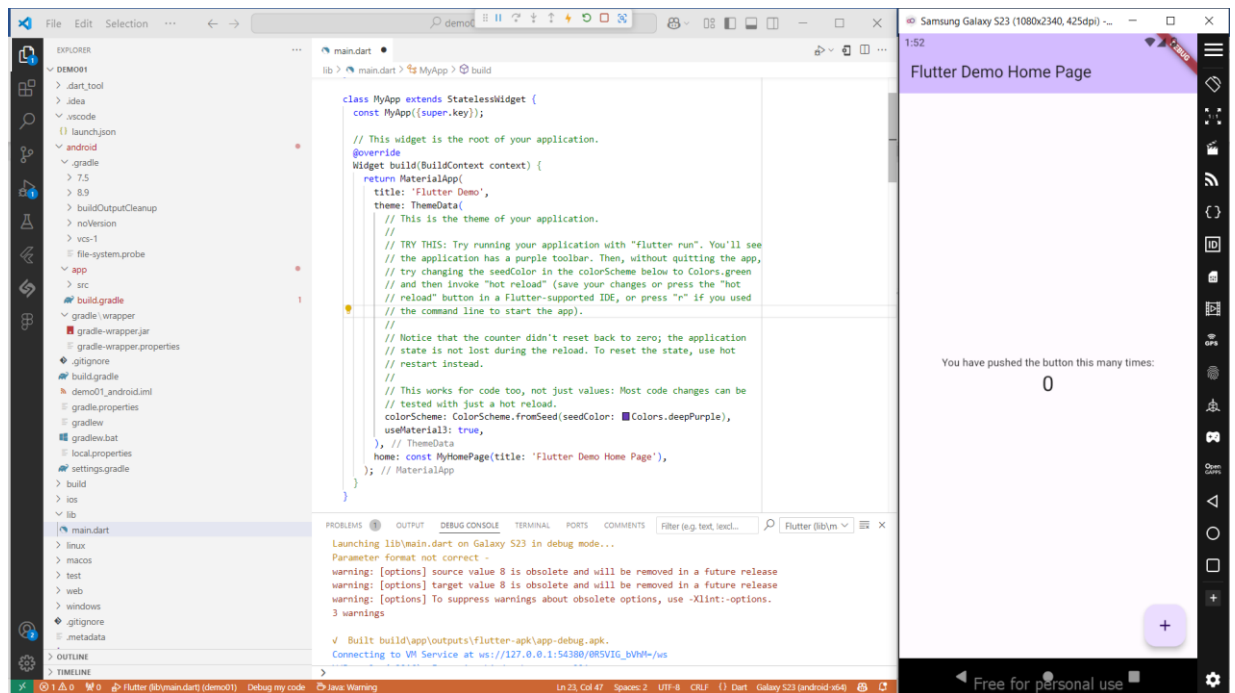
- Bấm vào nút Run để tạo máy ảo mà bạn cần. Ví dụ: thực thi máy Samsung Galaxy S23. Sau khi máy ảo (Emulator) thực thi ta có thể sắp xếp màn hình như sau:



- Bạn cần cài thêm plugin Extensions (ví dụ: Android iOS Emulator) để có thể thực thi các máy ảo trong VS Code.



- Thực thi trên máy ảo đã tạo



- Để tắt biểu tượng debug trên màn hình của giao diện ta có sử dụng câu lệnh sau:
`debugShowCheckedModeBanner: false`

c. Cấu trúc lưu trữ của một ứng dụng dùng flutter

my_flutter_app/	
├─ android/	# Thư mục dành cho mã nguồn Android
├─ ios/	# Thư mục dành cho mã nguồn iOS
├─ lib/	# Thư mục chứa mã nguồn Dart (chính của ứng dụng)
└─ main.dart	# Điểm khởi đầu của ứng dụng
└─ models/	# Thư mục chứa các lớp model (dữ liệu)
└─ screens/	# Thư mục chứa các màn hình (UI)
└─ widgets/	# Thư mục chứa các widget tùy chỉnh
└─ services/	# Thư mục chứa các dịch vụ (API, logic nghiệp vụ)
└─ utils/	# Thư mục chứa các tiện ích (helper, constants)
└─ ...	# Các thư mục khác tùy theo nhu cầu
└─ test/	# Thư mục chứa các tệp kiểm thử (unit test, widget test)
└─ assets/	# Thư mục chứa tài nguyên (hình ảnh, font chữ, JSON, v.v.)
└─ images/	# Thư mục chứa hình ảnh
└─ fonts/	# Thư mục chứa font chữ
└─ ...	# Các tài nguyên khác
└─ web/	# Thư mục dành cho mã nguồn web (nếu có)
└─ pubspec.yaml	# Tệp cấu hình dự án (dependencies, assets, v.v.)
└─ README.md	# Tệp mô tả dự án

Ví dụ: Xây dựng màn hình hiển thị thông tin của học phần lập trình di động với các nội dung như sau :



Hướng dẫn:

file main.dart

```
import 'package:demo01/screens/text_demo.dart';
import 'package:flutter/material.dart';

void main() {
  runApp(const TextDemo());
}
```

file text_demo.dart

```
import 'package:flutter/material.dart';

class TextDemo extends StatelessWidget {
  const TextDemo({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Ứng dụng demo Flutter',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
    );
  }
}
```

```
home: Scaffold(  
  appBar: AppBar(  
    title: const Text(  
      "Sử dụng Text và Image",  
      style: TextStyle(  
        color: Color.fromARGB(255, 201, 229, 16), fontSize: 18),  
      ),  
    backgroundColor: Colors.blue[900],  
    leading: IconButton(  
      icon: const Icon(Icons.home),  
      onPressed: () {},  
    ),  
  ),  
  body: Center(  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.start,  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        const SizedBox(height: 40),  
        const Padding(  
          padding: EdgeInsets.all(8.0),  
          child: Center(  
            child: Text("LẬP TRÌNH DI ĐỘNG KHÓA 13!",  
              style: TextStyle(  
                color: Color.fromARGB(255, 235, 19, 19),  
                fontSize: 25,  
                fontWeight: FontWeight.bold),  
                textAlign: TextAlign.center),  
            ),  
          ),  
        Center(  
          child: Image.asset("assets/images/logo_huit_01.png",  
            width: 200, height: 200, fit: BoxFit.scaleDown),  
        ),  
        const Padding(  
          padding: EdgeInsets.only(top: 30, left: 8),  
          child: Text("Chúc các bạn đạt kết quả tốt",  
            style: TextStyle(  
              color: Color.fromARGB(255, 169, 13, 248),  
              fontSize: 20,  
              fontWeight: FontWeight.bold),  
              textAlign: TextAlign.center),  
          ),  
        const SizedBox(height: 10),  
        const Padding(  
          padding: EdgeInsets.all(8.0),  
          child: Text("Giảng viên: Vũ Văn Vinh",  
            style: TextStyle(  
              color: Color.fromARGB(255, 54, 63, 244),  
              fontSize: 20,  
              fontWeight: FontWeight.bold),
```

```

        textAlign: TextAlign.left),
    ),
    const SizedBox(height: 10),
    const Padding(
      padding: EdgeInsets.all(8.0),
      child: Text("Số tiết: 75 tiết",
        style: TextStyle(
          color: Color.fromARGB(255, 54, 63, 244),
          fontSize: 20,
          fontWeight: FontWeight.bold),
        textAlign: TextAlign.left),
    ),
  ],
),
));
}
}

```

Một số widget :

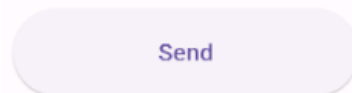
- **SizedBox** : là một widget trong Flutter được sử dụng để tạo ra một hộp có kích thước cố định. Nó rất hữu ích khi bạn cần đặt kích thước cụ thể cho một widget con hoặc tạo khoảng trống giữa các widget khác.

- **width**: Chiều rộng của SizedBox.
- **height**: Chiều cao của SizedBox.
- **child**: Widget con bên trong SizedBox (tùy chọn).

```

SizedBox(height: 50,width: 200,
  child: ElevatedButton(
    onPressed: () {}, child: const Text("Send")))

```



- **Padding**: là một widget được sử dụng để thêm khoảng trống (padding) xung quanh một widget con. Nó rất hữu ích khi bạn muốn tạo ra không gian giữa một widget và các widget xung quanh nó. Bạn có thể sử dụng các phương thức khác nhau của lớp EdgeInsets để xác định kích thước của khoảng trống này.

- **EdgeInsets.all**: tạo ra một khoảng trống đồng đều ở tất cả các cạnh
- **EdgeInsets.symmetric**: tạo ra một khoảng trống đồng đều ở tất cả các cạnh
- **EdgeInsets.only**: tạo ra một khoảng trống đồng đều ở tất cả các cạnh
- **EdgeInsets.fromLTRB**: tạo ra một khoảng trống đồng đều ở tất cả các cạnh

- **Padding**: là một widget trong Flutter được sử dụng để căn giữa một widget con trong không gian sẵn có. Nó rất hữu ích khi bạn muốn đặt một widget ở giữa theo cả chiều dọc và chiều ngang.

- **child**: Widget con mà bạn muốn căn giữa.

3. Bài tập tại lớp :

Bài tập 01. Xây dựng giao diện hiển thị thông tin của sinh viên, giảng viên khoa CNTT trường ĐH Công thương thành phố Hồ Chí Minh như sau :

Thông tin sinh viên	Thông tin sinh viên
	
Họ và tên: Nguyễn Văn A	Giảng viên Trần Thị A
MSSV: 2001221234	Khoa: Công nghệ Thông tin
Lớp: 13DHTH02	Học hàm: Thạc sỹ
Khóa: 13 Đại học	Chuyên ngành: CNPM
Ngành: Công nghệ thông tin	Giảng dạy: Nhập môn lập trình, Lập trình windows, Lập trình we...
Trường: Đại học Công Thương Thành phố Hồ Chí Minh	
Trở về	Trở về

Bài tập 02. Xây dựng giao diện hiển thị thông tin của một đề tài đồ án khóa luận gồm các thông tin : mã đề tài đồ án, tên đề tài, số lượng sinh viên tối đa, chuyên ngành, giảng viên hướng dẫn, yêu cầu đề tài.

Bài tập 03. Xây dựng giao diện hiển thị thông tin của một sản phẩm trong cửa hàng : 03 hình ảnh về sản phẩm đó, mã sản phẩm, tên sản phẩm, nhà sản xuất, giá bán, mô tả sản phẩm.

4. Bài tập về nhà :

Bài tập 04. Xây dựng giao diện hiển thị thông tin của một nhóm gồm : mã nhóm, tên nhóm, số lượng thành viên, thông tin của 3 thành viên : mã sinh viên, tên sinh viên, vai trò (nhóm trưởng, thành viên).

Bài tập 05. Xây dựng giao diện giới thiệu thông tin về ngành học CNTT và ngành học ATTT tại khoa Công nghệ Thông tin trường ĐH Công thương Thành phố Hồ Chí Minh.